

claude-windows / 2ba49afa-99f4-41f2-86a2-27a289d432f3

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\2ba49afa-99f4-41f2-86a2-27a289d432f3\subagents\workflows\wf_d954e9ee-fd0\agent-a0d6066e15ec1067b.jsonl`
- SHA-256: `e2b9b3bfe777b64f9b8a0b3d5bc41f629b45c88bf69302f55ad9427be8c7a01e`
- Source modified: `2026-06-21T15:18:21+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf_d954e9ee-fd0`
- session_id: `2ba49afa-99f4-41f2-86a2-27a289d432f3`
```

Transcript

1. user (2026-06-21T15:13:32.457Z)

You are reviewing for FAITHFULNESS to the A5 spec + OBSERVABILITY. A5 refactor under C:/Users/avidu/Projects/khelsutra/web/src: a shared error->text mapping.

- NEW lib/errorText.ts: errorText(t, e) ? engine code -> e.detail ?? t("errors.engineGeneric"); else t(`errors.\${e.code}`).
- App.tsx onBuild catch now: classifyError(err) -> errorText(t, e) (was: parseDetail(err.body) ?? String(err)). parseDetail import removed.
- IngestPanel.tsx: errorMsg now = errorText(t, phase.error) (was: inline ingest.\${code} map).
- Catalogs (en/kn/hi): MOVED ingest.{networkError,invalidUri,emptyResult,timeout} -> new errors.* namespace; ADDED errors.engineGeneric; REMOVED ingest.failed. ingest.* kept: title/intro/uriPlaceholder/uriAriaLabel/hint/submit/submitted/cancel/working/progress/retry.

Tests: 135 green incl. errorText.test.ts, build-error.test.tsx, the existing IngestPanel.test.tsx, i18n key-parity.

A5 spec: "Friendly engine-error mapping over raw FastAPI detail (SPA, client-side map)" ? reuse classifyError on the build path; add a shared errors.* namespace; recommend unifying IngestPanel onto it.

Check:

1. Is the build path (App.onBuild -> CompileResult) now friendly + LOCALIZED (uses t), not a raw String(err)/HTTP-500? Read App.tsx + components/CompileResult.tsx.
 2. Is the errors.* namespace genuinely SHARED (both IngestPanel and App use errorText)? Any duplication left?
 3. OBSERVABILITY (owner bar: loud, structured logs; correlation ids): does the build.failed log still carry the cid + now code/detail/status so a failure is debuggable? Is anything dropped vs before?
 4. Is errorText correctly placed OUT of the i18n-free errors.ts (so errors.ts stays testable)? Any layering smell?
 5. engineGeneric copy: is it a sensible last-resort (engine error with no detail)? Could it ever swallow a useful message?
- Be precise. Empty findings if it faithfully meets A5.

2. assistant (2026-06-21T15:13:36.334Z)

I'll review the A5 refactor for faithfulness to the spec and observability. Let me start by reading the relevant files.

3. user (2026-06-21T15:13:39.619Z)

Already up to date.
---BRANCH---
feat/a5-friendly-build-errors

4. user (2026-06-21T15:13:39.633Z)

```
1      // The ONE place a classified error (lib/errors.ts) becomes user-facing text. Both
```

surfaces use it:

```
2      // the ingest flow (IngestPanel) and the build flow (App.onBuild ? CompileResult), so a
5xx / engine-
3      // down failure reads the same friendly way everywhere instead of a raw "HTTP 500" /
fetch message.
4      //
5      // `engine` carries the engine's OWN message (the most specific) ? shown verbatim, never
translated.
6      // Every other code maps to a friendly localized line in the shared `errors.*`
namespace. Kept out of
7      // errors.ts so that module stays i18n-free and unit-testable without a translation
context.
8      import type { TFunction } from "i18next";
9      import type { IngestError } from "../errors";
10
11     export function errorText(t: TFunction, e: IngestError): string {
12         if (e.code === "engine") return e.detail ?? t("errors.engineGeneric");
13         return t(`errors.${e.code}`);
14     }
15
```

5. user (2026-06-21T15:13:39.643Z)

```
1      // Error helpers shared by the ingest flow and the build flow. Kept i18n-FREE: callers
map an
2      // IngestErrorCode to a localized string (so hooks stay testable without a translation
context),
3      // while the engine's own `detail` text passes through verbatim (it's the most specific
message).
4      import { ApiError } from "../api/client";
5
6      /** Pull FastAPI's {"detail": "..."} message out of an error body, else return the raw
body. */
7      export function parseDetail(body: string): string {
8          try {
9              const j = JSON.parse(body) as { detail?: unknown };
10             if (j && typeof j.detail === "string") return j.detail;
11         } catch {
12             /* not JSON ? fall through to the raw body */
13         }
14         return body;
15     }
16
17     /** Stable, localizable failure categories surfaced by the ingest flow.
18     * `engine` carries the engine's own message in `detail`; the rest are localized by
code. */
19     export type IngestErrorCode = "invalidUri" | "networkError" | "emptyResult" | "timeout"
| "engine";
20
21     export interface IngestError {
22         code: IngestErrorCode;
23         /** Present for `engine` ? the verbatim FastAPI detail / status; rendered as-is, not
translated. */
24         detail?: string;
25     }
26
27     /** Classify a thrown client error into a stable code. A network throw (ApiError with no
`status`)
28     * is the common "engine unreachable" case ? its own friendly code; any HTTP error
surfaces the
29     * engine's `detail` (e.g. an unsupported-scheme 400, a not-found 404). */
30     export function classifyError(err: unknown): IngestError {
31         if (err instanceof ApiError) {
32             if (err.status === undefined) return { code: "networkError" }; // fetch threw ?
engine truly unreachable

```

```

33     if (err.body) {
34         const detail = parseDetail(err.body);
35         if (detail) return { code: "engine", detail }; // a real engine message (e.g. a
400 unsupported-scheme)
36     }
37     // A bare 5xx with no body is almost always the engine being down / a gateway that
can't reach it
38     // (e.g. the dev proxy when :8000 is off) ? for a non-technical user a friendly "is
it running?"
39     // beats a cryptic "HTTP 500". A 4xx with no body keeps the status (it's a client-
side mistake).
40     if (err.status >= 500) return { code: "networkError" };
41     return { code: "engine", detail: `HTTP ${err.status}` };
42 }
43 return { code: "engine", detail: String(err) };
44 }
45

```

6. assistant (2026-06-21T15:13:43.657Z)

Now let me read App.tsx, CompileResult.tsx, IngestPanel.tsx, and the catalogs.

7. user (2026-06-21T15:13:44.616Z)

```

1     import { useEffect, useMemo, useState } from "react";
2     import { useTranslation, Trans } from "react-i18next";
3     import { useRallySelection } from "../hooks/useRallySelection";
4     import { RallyGrid } from "../components/RallyGrid";
5     import { QueryBar } from "../components/QueryBar";
6     import { SelectionFooter } from "../components/SelectionFooter";
7     import { LanguageSwitcher } from "../components/LanguageSwitcher";
8     import { CompileResult, type BuildState } from "../components/CompileResult";
9     import { IngestPanel } from "../components/IngestPanel";
10    import { runQuery, type ParsedQuery } from "../lib/query";
11    import { floorRisk } from "../lib/floor";
12    import { reelFilename } from "../lib/format";
13    import { classifyError } from "../lib/errors";
14    import { errorText } from "../lib/errorText";
15    import { log, newCorrelationId } from "../lib/log";
16    import { ApiError, compileReel, getConfig, highlightUrl, listRallies, minShotCount }
from "../api/client";
17    import type { EngineConfig, RallySegment } from "../api/types";
18
19    /** One compact, structured description of how a query was understood (for the trace
log). */
20    function parseTrace(q: ParsedQuery) {
21        return {
22            raw: q.raw,
23            recognized: q.recognized,
24            filters: q.filters.map((f) => `${f.field}${f.op}${f.value}`),
25            sort: q.sort ? `${q.sort.key}:${q.sort.dir}` : null,
26            limit: q.limit ?? null,
27            unavailable: q.unavailable.map((u) => `${u.kind}:${u.token}`),
28        };
29    }
30
31    // Demo data shown when no ?video=<id> is supplied. Shape matches the verified
play_segments
32    // contract; only honest fields are surfaced. A real session loads from POST /api/query.
33    const DEMO_RALLIES: RallySegment[] = [
34        { id: 1, start_time: 12, end_time: 20, duration: 8, server: null, receiver: null,
metadata: { shot_count: 9, shot_count_confidence: "High" } },
35        { id: 2, start_time: 41, end_time: 47, duration: 6, server: null, receiver: null,
metadata: { shot_count: 5 } },
36        { id: 3, start_time: 63, end_time: 84, duration: 21, server: null, receiver: null,

```

```

metadata: { shot_count: 19, shot_count_confidence: "Medium" } },
37     { id: 4, start_time: 95, end_time: 101, duration: 6, server: null, receiver: null,
metadata: { } },
38     { id: 5, start_time: 130, end_time: 142, duration: 12, server: null, receiver: null,
metadata: { shot_count: 11 } },
39     { id: 6, start_time: 158, end_time: 187, duration: 29, server: null, receiver: null,
metadata: { shot_count: 24, shot_count_confidence: "High" } },
40 ];
41
42 type LoadState =
43   | { status: "demo" }
44   | { status: "loading" }
45   | { status: "loaded"; count: number }
46   | { status: "error"; error: string };
47
48 export default function App() {
49   const { t } = useTranslation();
50   // Job-tethered (D10): a finished job's video_id arrives via ?video=<id> ? seeded from
the URL,
51   // but now also SETTABLE so the in-UI ingest panel (B-P2) can hand off a freshly-
processed video
52   // without a reload (it also rewrites ?video= so a refresh is stable).
53   const [videoId, setVideoId] = useState<string | null>(
54     () => new URLSearchParams(window.location.search).get("video"),
55   );
56
57   const [rallies, setRallies] = useState<RallySegment[]>(videoId ? [] : DEMO_RALLIES);
58   const [order, setOrder] = useState<RallySegment[]>(videoId ? [] : DEMO_RALLIES);
59   const [loadState, setLoadState] = useState<LoadState>(videoId ? { status: "loading" }
: { status: "demo" });
60   const [reelName, setReelName] = useState("highlights");
61   const [build, setBuild] = useState<BuildState>({ status: "idle" });
62
63   // Display order is separate from selection: a query can re-sort the cards while the
shared
64   // selection set (sel) stays the single source of truth for what lands in the reel.
65   const sel = useRallySelection(rallies, "all");
66
67   // Load real rallies for a job's video (POST /api/query). Default ALL-selected (D8).
68   useEffect(() => {
69     if (!videoId) return;
70     const cid = newCorrelationId();
71     log("info", "rallies.load_start", { cid, video_id: videoId });
72     // Show "loading" even when videoId was set dynamically by the ingest panel (initial
state was
73     // "demo" in that path) ? the banner must reflect the load, not stale demo copy.
74     setLoadState({ status: "loading" });
75     let alive = true;
76     listRallies(videoId, cid)
77       .then((rs) => {
78         if (!alive) return;
79         setRallies(rs);
80         setOrder(rs);
81         sel.apply(rs.map((r) => r.id));
82         setLoadState({ status: "loaded", count: rs.length });
83         log("info", "rallies.loaded", { cid, video_id: videoId, count: rs.length });
84       })
85       .catch((err) => {
86         if (!alive) return;
87         setLoadState({ status: "error", error: String(err) });
88         log("error", "rallies.load_failed", { cid, video_id: videoId, error: String(err)
});
89       });
90     return () => {
91       alive = false;

```

```

92     };
93     // videoId is stable for the page; sel.apply is a stable callback.
94     // eslint-disable-next-line react-hooks/exhaustive-deps
95     }, [videoId]);
96
97     // Read the engine's stitching.min_shot_count so the footer can WARN before a selected
rally is
98     // silently dropped at compile (P4 / D7). Offline (no engine) is fine: no config ? no
warning.
99     const [cfg, setCfg] = useState<EngineConfig | null>(null);
100    useEffect(() => {
101        let alive = true;
102        getConfig()
103            .then((c) => alive && setCfg(c))
104            .catch((err) => log("warn", "config.unavailable", { error: String(err) }));
105        return () => {
106            alive = false;
107        };
108    }, []);
109    const warning = floorRisk(rallies, sel.selected, cfg ? minShotCount(cfg) : undefined);
110
111    // Stable chronological ordinal (by start_time) so re-sorting never rennumbers a rally.
112    const ordinalOf = useMemo(() => {
113        const m = new Map<number, number>();
114        [...rallies].sort((a, b) => a.start_time - b.start_time).forEach((r, i) =>
m.set(r.id, i + 1));
115        return (id: number) => m.get(id) ?? 0;
116    }, [rallies]);
117
118    // Build ? POST /api/compile ? preview/download the stitched reel. One cid traces the
whole call.
119    const onBuild = async () => {
120        if (build.status === "building") return;
121        const cid = newCorrelationId();
122        const filename = reelFilename(reelName);
123        const ids = [...sel.selected];
124        if (!videoId) {
125            log("info", "build.no_video", { cid, selected: ids.length });
126            setBuild({
127                status: "error",
128                error: t("build.noVideo"),
129            });
130            return;
131        }
132        setBuild({ status: "building" });
133        log("info", "build.start", {
134            cid,
135            video_id: videoId,
136            filename,
137            selected: ids.length,
138            total_sec: Math.round(sel.totalSec),
139            below_floor: warning?.atRisk.length ?? 0,
140        });
141        try {
142            const res = await compileReel(videoId, ids, filename, cid);
143            const url = highlightUrl(videoId, filename);
144            setBuild({ status: "done", url, filename });
145            log("info", "build.done", { cid, video_id: videoId, filename, filepath:
res.filepath, url });
146        } catch (err) {
147            // Same friendly mapping the ingest flow uses: a bare 5xx / network throw reads
"is the engine
148            // running?", a real engine 4xx surfaces its own detail ? never a raw "HTTP 500"
(A5).
149            const status = err instanceof ApiError ? err.status : undefined;

```

```

150         const e = classifyError(err);
151         setBuild({ status: "error", error: errorText(t, e) });
152         log("error", "build.failed", { cid, video_id: videoId, filename, status, code:
e.code, detail: e.detail, error: String(err) });
153     }
154 };
155
156     // Every query application is ONE correlatable journey: a `query.parse` line records
exactly how
157     // the text was understood, then a single outcome line (apply / skip / empty) shares
the same cid.
158     // Unsupported concepts and zero-result queries are logged loudly ? never silently
dropped.
159     const onApplyQuery = (q: ParsedQuery, source: "typed" | "example") => {
160         const cid = newCorrelationId();
161         log("info", "query.parse", { cid, source, ...parseTrace(q) });
162
163         // Demand signal + honest "we ignored this": surface every shot-type/player/score
the user asked
164         // for but we can't yet honour (PER_RALLY_SELECTION.md §3 roadmap fields).
165         if (q.unavailable.length > 0) {
166             log("info", "query.unavailable_request", { cid, source, raw: q.raw, unavailable:
q.unavailable });
167         }
168
169         if (!q.recognized) {
170             log("info", "query.skip", { cid, source, raw: q.raw, reason: "no actionable
clause" });
171             return; // empty / unparseable ? leave the user's curation untouched
172         }
173
174         try {
175             const { ordered, selectedIds } = runQuery(rallies, q);
176             setOrder(ordered);
177             sel.apply(selectedIds);
178             log("info", "query.apply", {
179                 cid,
180                 source,
181                 filters: q.filters.length,
182                 sort: q.sort ? `${q.sort.key}:${q.sort.dir}` : null,
183                 limit: q.limit ?? null,
184                 total: rallies.length,
185                 selected: selectedIds.length,
186                 selected_ids: selectedIds,
187                 dropped_unavailable: q.unavailable.map((u) => `${u.kind}:${u.token}`),
188             });
189             if (selectedIds.length === 0) {
190                 // Loud: a recognized query that matches nothing makes the grid look empty ? say
why.
191                 log("warn", "query.empty_result", { cid, source, ...parseTrace(q) });
192             }
193         } catch (err) {
194             // Defensive: the pure parser/selector shouldn't throw, but never fail silently if
it does.
195             log("error", "query.apply_failed", { cid, source, raw: q.raw, error: String(err)
});
196         }
197     };
198
199     // Manual curation edits are traced too, so a journey (query ? hand-tune) reads as one
story.
200     const onToggle = (id: number) => {
201         log("debug", "rally.toggle", { id, action: sel.isSelected(id) ? "remove" : "add" });
202         sel.toggle(id);
203     };

```

```

204     const onBulk = (op: "select_all" | "clear" | "invert") => {
205         const resultCount = op === "select_all" ? rallies.length : op === "clear" ? 0 :
rallies.length - sel.count;
206         log("info", "selection.bulk", { cid: newCorrelationId(), op, result_count:
resultCount });
207         if (op === "select_all") sel.selectAll();
208         else if (op === "clear") sel.clearAll();
209         else sel.invert();
210     };
211
212     return (
213         <main className="min-h-screen bg-paper text-ink pb-24">
214             <header className="bg-ink text-white">
215                 <div className="mx-auto flex max-w-5xl items-center justify-between gap-3 px-6
py-4">
216                     <span className="text-xl font-extrabold tracking-tight">
217                         Khel<span className="text-green">?</span>Sutra <span className="font-
semibold text-lime">Studio</span>
218                     </span>
219                     <LanguageSwitcher />
220                 </div>
221             </header>
222
223             <section className="mx-auto max-w-5xl px-6 py-10">
224                 <span className="inline-flex rounded-full bg-green/15 px-3 py-1 text-xs font-
semibold text-green">
225                     {t("app.badge")}
226                 </span>
227                 <h1 className="mt-4 text-3xl font-extrabold tracking-
tight">{t("app.title")}</h1>
228                 <p className="mt-2 max-w-2xl text-ink/70">
229                     <Trans i18nKey="app.intro" components={{ b: <strong />, i: <em /> }} />
230                 </p>
231
232                 /* Front-half ingest (B-P2): when no video is loaded, point the tool at a
bucket URI / host
233                     path ? process ? load. Once a video_id exists, the existing back-half takes
over. */
234                 {!videoId && (
235                     <IngestPanel
236                         onLoad={id => {
237                             setVideoId(id);
238                             window.history.replaceState(null, "", `?video=${encodeURIComponent(id)}`);
239                         }}
240                     />
241                 )}
242
243                 <ModeBanner load={loadState} videoId={videoId} />
244
245                 <div className="mt-6">
246                     <QueryBar onApply={onApplyQuery} />
247                 </div>
248
249                 <div className="mt-6 flex flex-wrap gap-2 text-sm">
250                     <button type="button" onClick={() => onBulk("select_all")} className="inline-
flex min-h-11 items-center rounded-full border border-black/15 px-4 hover:border-
green">{t("bulk.selectAll")}</button>
251                     <button type="button" onClick={() => onBulk("clear")} className="inline-flex
min-h-11 items-center rounded-full border border-black/15 px-4 hover:border-
green">{t("bulk.clear")}</button>
252                     <button type="button" onClick={() => onBulk("invert")} className="inline-flex
min-h-11 items-center rounded-full border border-black/15 px-4 hover:border-
green">{t("bulk.invert")}</button>
253                 </div>
254

```

```

255         <div className="mt-5">
256             <RallyGrid rallies={order} isSelected={sel.isSelected} onToggle={onToggle}
ordinalOf={ordinalOf} />
257         </div>
258
259         <CompileResult build={build} onDismiss={() => setBuild({ status: "idle" })} />
260     </section>
261
262     <SelectionFooter
263         count={sel.count}
264         totalSec={sel.totalSec}
265         reelName={reelName}
266         onReelNameChange={setReelName}
267         onBuild={onBuild}
268         building={build.status === "building"}
269         warning={warning}
270     />
271 </main>
272 );
273 }
274
275 function ModeBanner({ load, videoId }: { load: LoadState; videoId: string | null }) {
276     if (load.status === "demo") {
277         return (
278             <p className="mt-3 text-sm text-ink/50" data-testid="mode-banner">
279                 <Trans i18nKey="mode.demo" components={{ code: <code /> }} />
280             </p>
281         );
282     }
283     if (load.status === "loading") {
284         return (
285             <p className="mt-3 text-sm text-ink/60" data-testid="mode-banner">
286                 <Trans i18nKey="mode.loading" values={{ videoId }} components={{ code: <code />
}} />
287             </p>
288         );
289     }
290     if (load.status === "error") {
291         return (
292             <p className="mt-3 rounded-lg border border-red-200 bg-red-50 px-3 py-2 text-sm
text-red-800" data-testid="mode-banner">
293                 <Trans i18nKey="mode.error" values={{ videoId, error: load.error }}
components={{ code: <code /> }} />
294             </p>
295         );
296     }
297     return (
298         <p className="mt-3 text-sm text-ink/50" data-testid="mode-banner">
299             <Trans i18nKey="mode.loaded" values={{ count: load.count }} components={{ b:
<strong className="text-ink/70" /> }} />
300         </p>
301     );
302 }
303

```

8. user (2026-06-21T15:13:45.107Z)

```

1      // The compiled-reel result (PER_RALLY_SELECTION.md P5): once Build ? POST /api/compile
succeeds,
2      // preview the reel inline (<video> over GET /api/highlights/...) and offer a download.
The engine
3      // returns a server-local filepath, NOT a URL (main.py:394), so the SPA builds the URL
itself. On a
4      // failure (e.g. the min_shot_count 400) we show the engine's reason ? loud, never
swallowed.

```



```

5
6   import { useTranslation } from "react-i18next";
7
8   export type BuildState =
9     | { status: "idle" }
10    | { status: "building" }
11    | { status: "done"; url: string; filename: string }
12    | { status: "error"; error: string };
13
14   interface Props {
15     build: BuildState;
16     onDismiss: () => void;
17   }
18
19   export function CompileResult({ build, onDismiss }: Props) {
20     const { t } = useTranslation();
21     if (build.status === "idle" || build.status === "building") return null;
22
23     return (
24       <section
25         data-testid="compile-result"
26         className={
27           "mt-6 rounded-xl border p-4 " +
28           (build.status === "error" ? "border-red-200 bg-red-50" : "border-green/30 bg-
green/5")
29         >
30         <div className="flex items-start justify-between gap-3">
31           <h2 className="font-semibold text-ink">
32             {build.status === "done" ? t("result.ready") : t("result.failed")}
33           </h2>
34           <button
35             type="button"
36             onClick={onDismiss}
37             aria-label={t("result.dismiss")}
38             className="flex h-11 w-11 shrink-0 items-center justify-center rounded-full
text-ink/50 hover:text-ink"
39           >
40             ?
41           </button>
42         </div>
43
44         {build.status === "done" ? (
45           <div className="mt-3">
46             /* eslint-disable-next-line jsx-a11y/media-has-caption ? a stitched rally
reel has no captions */
47             <video
48               data-testid="reel-video"
49               src={build.url}
50               controls
51               playsInline
52               className="w-full max-w-xl rounded-lg bg-black"
53             />
54             <a
55               href={build.url}
56               download={build.filename}
57               className="mt-3 inline-flex min-h-11 items-center rounded-full bg-green px-4
py-2 text-sm font-semibold text-ink/50"
58             >
59               {t("result.download", { filename: build.filename })}
60             </a>
61           </div>
62         ) : (
63           <p className="mt-2 text-sm text-red-800" data-testid="compile-error">
64             {build.error}
65

```

```

66         </p>
67     })
68 </section>
69 );
70 }
71

```

9. user (2026-06-21T15:13:45.685Z)

```

1      // The front-half ingest screen (appliance S1/S2, tracker B-P2). Point the tool at a
video ? a
2      // `gs://` bucket URI or a path on the engine host ? and it submits to POST
/api/process, shows
3      // live progress while polling GET /api/jobs/{id}, and hands the resulting video_id to
the existing
4      // rally-selection back-half (App lifts ?video=<id>). Honest by construction: the engine
reports
5      // coarse phase WORDS (not a %), and a success-shaped failure (done but no rallies) is
shown, never
6      // silently treated as a load.
7      import { useState } from "react";
8      import type { FormEvent } from "react";
9      import { useTranslation } from "react-i18next";
10     import { useIngestJob } from "../hooks/useIngestJob";
11     import { errorText } from "../lib/errorText";
12
13     interface Props {
14         /** Called with the engine's video_id once a submitted job finishes successfully. */
15         onLoaded: (videoId: string) => void;
16     }
17
18     export function IngestPanel({ onLoaded }: Props) {
19         const { t } = useTranslation();
20         const [uri, setUri] = useState("");
21         const { phase, submit, cancel, reset, busy } = useIngestJob(onLoaded);
22
23         const onSubmit = (e: FormEvent) => {
24             e.preventDefault();
25             submit(uri);
26         };
27
28         // Shared mapping with the build flow (lib/errorText): `engine` failures carry the
engine's own
29         // message (a 400 scheme/path detail, a worker error); every other code maps to a
friendly
30         // localized line ? the network case especially, since this is the first screen a
fresh user hits
31         // and "is the engine running?" must read plainly.
32         const errorMsg = phase.status === "error" ? errorText(t, phase.error) : null;
33
34         return (
35             <section className="mt-6 rounded-2xl border border-black/10 bg-white p-5" data-
testid="ingest-panel">
36                 <h2 className="text-lg font-bold">{t("ingest.title")}</h2>
37                 <p className="mt-1 max-w-2xl text-sm text-ink/60">{t("ingest.intro")}</p>
38
39                 <form onSubmit={onSubmit} className="mt-3 flex flex-wrap items-center gap-2">
40                     <input
41                         type="text"
42                         value={uri}
43                         onChange={(e) => setUri(e.target.value)}
44                         placeholder={t("ingest.uriPlaceholder")}
45                         aria-label={t("ingest.uriAriaLabel")}
46                         disabled={busy}
47                         className="min-h-11 min-w-[18rem] flex-1 rounded-xl border border-black/15 bg-

```

```

white px-4 py-2 text-sm text-ink outline-none focus:border-green disabled:opacity-50"
48         />
49         <button
50             type="submit"
51             disabled={busy}
52             className="min-h-11 rounded-xl bg-ink px-4 py-2 text-sm font-semibold text-
white disabled:opacity-40"
53         >
54             {phase.status === "submitting" ? t("ingest.submitting") : t("ingest.submit")}
55         </button>
56         {busy && (
57             <button
58                 type="button"
59                 onClick={cancel}
60                 className="min-h-11 rounded-xl border border-black/15 px-4 py-2 text-sm
hover:border-green"
61             >
62                 {t("ingest.cancel")}
63             </button>
64         )}
65     </form>
66
67     <p className="mt-2 text-xs text-ink/40">{t("ingest.hint")}</p>
68
69     {/* Live status (aria-live so screen readers hear progress). Indeterminate: the
engine reports
70         phase words ("hashing", "segmenting"), not a percentage ? so a spinner + the
phase word. */}
71     <div aria-live="polite" className="mt-3 text-sm" data-testid="ingest-status">
72         {busy && (
73             <p className="flex items-center gap-2 text-ink/70">
74                 <Spinner />
75                 <span>
76                     {t("ingest.working")}
77                     {phase.status === "polling" && phase.progress
78                         ? ` · ${t("ingest.progress", { progress: phase.progress })}`
79                         : ""}
80                 </span>
81             </p>
82         )}
83         {phase.status === "error" && (
84             <div className="rounded-lg border border-red-200 bg-red-50 px-3 py-2 text-
red-800" data-testid="ingest-error">
85                 <p>{errorMsg}</p>
86                 <button
87                     type="button"
88                     onClick={reset}
89                     className="mt-2 inline-flex min-h-11 items-center rounded-lg border
border-red-300 px-3 text-sm hover:bg-red-100"
90                 >
91                     {t("ingest.retry")}
92                 </button>
93             </div>
94         )}
95     </div>
96 </section>
97 );
98 }
99
100 function Spinner() {
101     return (
102         <span
103             className="inline-block h-3.5 w-3.5 animate-spin rounded-full border-2 border-
ink/20 border-t-ink/70"
104             aria-hidden="true"

```

```

105         />
106     );
107 }
108

```

10. assistant (2026-06-21T15:13:49.434Z)

Now let me look at the catalogs (en/kn/hi), the log lib, and check the previous behavior via git diff.

11. user (2026-06-21T15:13:50.129Z)

```

web\src\i18n\locales\en\common.json
web\src\i18n\locales\kn\common.json
web\src\i18n\locales\hi\common.json

```

12. user (2026-06-21T15:13:50.611Z)

```

web\src\i18n\index.ts
web\src\i18n\hygiene.test.ts
web\src\i18n\locales\en\common.json
web\src\i18n\locales\kn\common.json
web\src\i18n\locales\hi\common.json

```

13. user (2026-06-21T15:13:51.116Z)

```

1      // Structured client logger ? the observability seed for the operator console
2      // (docs/LOCAL_APPLIANCE_ARCHITECTURE.md §5). Every API call carries a CORRELATION ID so
a
3      // single user journey is traceable across the UI ? FastAPI ? worker. Logs are JSON-
shaped
4      // so they aggregate cleanly off a transient host. Keep this dependency-free.
5
6      export type LogLevel = "debug" | "info" | "warn" | "error";
7
8      /** A short, unique id to correlate all log lines (and the x-correlation-id header) for
one action. */
9      export function newCorrelationId(): string {
10         const c = (globalThis as { crypto?: Crypto }).crypto;
11         if (c && typeof c.randomUUID === "function") return c.randomUUID();
12         // Fallback for environments without crypto.randomUUID.
13         return `cid_${Date.now().toString(36)}_${Math.floor(Math.random() *
1e9).toString(36)}`;
14     }
15
16     export function log(level: LogLevel, event: string, fields: Record<string, unknown> =
{}): void {
17         const line = JSON.stringify({ ts: new Date().toISOString(), level, event, ...fields
});
18         if (level === "error") console.error(line);
19         else if (level === "warn") console.warn(line);
20         else console.log(line);
21     }
22

```

14. user (2026-06-21T15:13:54.681Z)

```

1      {
2          "app": {
3              "badge": "pick your rallies · alpha",
4              "title": "Pick the rallies for your reel",
5              "intro": "Filter by <b>rally length</b> ? e.g. <i>?top 15 longest?</i> ? then fine-
tune by tapping cards. Everything starts <b>selected</b>; just drop the ones you don't want."
6          },

```

```

7      "lang": {
8          "label": "Language",
9          "en": "English",
10         "kn": "?????",
11         "hi": "?????"
12     },
13     "bulk": {
14         "selectAll": "Select all",
15         "clear": "Clear",
16         "invert": "Invert"
17     },
18     "mode": {
19         "demo": "These are sample rallies ? process a video above to work on a real match.",
20         "loading": "Finding the rallies in your video?",
21         "error": "Couldn't load this video's rallies: {error}",
22         "loaded": "Found <b>{count}</b> rallies in your video."
23     },
24     "query": {
25         "placeholder": "e.g. over 10s · top 15 longest · shortest first",
26         "ariaLabel": "Filter rallies by length, count or order",
27         "apply": "Apply",
28         "notAvailable": "not available yet",
29         "notAvailableTitle": "?{token}? needs {kind} detection ? not available yet",
30         "notAvailableHelp": "Shot type, players and score aren't available yet ? filter by
rally length instead.",
31         "unreadable": "Couldn't read that ? try ?over 10s? or ?top 15 longest?.",
32         "hint": "Type over 10s for rallies longer than 10 seconds ? or tap a sample below.",
33         "try": "Sample queries:",
34         "support": "Filtering by <b>rally length</b> is fully supported. Shot-count is
<x>experimental</x> (under development).",
35     },
36     "grid": {
37         "empty": "No rallies to show yet.",
38         "rally": "Rally {n}",
39         "inReel": "Rally {n}, in reel",
40         "notInReel": "Rally {n}, not in reel",
41         "start": "start",
42         "long": "long",
43         "shots": "~{count} shots · experimental",
44         "shotsTitle": "Shot count is experimental (under development) and may be
inaccurate{conf}. Filter by rally length instead.",
45         "shotsConfidence": " ? confidence: {conf}"
46     },
47     "footer": {
48         "count": "{count, plural, one {# rally} other {# rallies}}",
49         "summaryRest": "selected · {total} total",
50         "reelName": "Reel name",
51         "reelNamePlaceholder": "my-highlights",
52         "build": "Build reel ({count})",
53         "building": "Building?",
54         "floorOne": "<b>Your only selected rally</b> has fewer than {min} shots ? it would
be skipped, so the build would fail. Pick a longer rally.",
55         "floorAll": "<b>All {count} selected rallies</b> have fewer than {min} shots ?
they'd all be skipped, so the build would fail. Pick longer rallies.",
56         "floorSomeLead": "{count, plural, one {# rally} other {# rallies}}",
57         "floorSomeRest": "with fewer than {min} shots will be skipped when building (they're
too short).",
58     },
59     "result": {
60         "ready": "Your reel is ready",
61         "failed": "Build failed",
62         "dismiss": "Dismiss",
63         "download": "Download {filename}"
64     },
65     "build": {

```

```

66         "noVideo": "These are sample rallies ? process a real video first to build a reel."
67     },
68     "ingest": {
69         "title": "Process a video",
70         "intro": "Point KhelSutra at a video and it finds the rallies for you ? this can
take a few minutes.",
71         "uriPlaceholder": "gs://your-bucket/match.mp4",
72         "uriAriaLabel": "Video location ? a gs:// bucket URL or a path on the engine host",
73         "hint": "Paste a cloud-storage link (gs://?), or a path to a video on the machine
running KhelSutra.",
74         "submit": "Process video",
75         "submitting": "Submitting?",
76         "cancel": "Cancel",
77         "working": "Processing your video?",
78         "progress": "{progress}",
79         "retry": "Try again"
80     },
81     "errors": {
82         "networkError": "Can't reach KhelSutra. Is it running?",
83         "invalidUri": "Enter a video location first ? a gs:// link or a path on the machine
running KhelSutra.",
84         "emptyResult": "Processing finished but found no rallies in this video. Try a
clearer match recording.",
85         "timeout": "This is taking longer than expected. The job may still be running ?
reload with the video link, or try again.",
86         "engineGeneric": "Something went wrong. Please try again."
87     },
88     "selection": {
89         "count": "{count, plural, one {# rally selected} other {# rallies selected}}"
90     }
91 }
92

```

15. user (2026-06-21T15:13:56.242Z)

---DIFF STAT vs master---

16. assistant (2026-06-21T15:14:00.035Z)

The diff vs origin/master is empty, which means the branch may not have been pushed or commits aren't yet made. Let me check the actual working state and git status.

17. user (2026-06-21T15:14:02.324Z)

===STATUS===

```

M web/src/App.tsx
M web/src/components/IngestPanel.tsx
A web/src/components/build-error.test.tsx
M web/src/i18n/locales/en/common.json
M web/src/i18n/locales/hi/common.json
M web/src/i18n/locales/kn/common.json
A web/src/lib/errorText.test.ts
A web/src/lib/errorText.ts
?? .claude/
?? site/access.db

```

===LOG vs master (local)===

===HEAD===

```

95195a5 feat(web): A4 ? mobile touch targets (?44px) + visible tooltip text (#61)
ccdcfd feat(web): A1 ? plain-language copy pass on the en catalog (de-jargon) (#60)
a20002f docs: propose Multilingual Expansion ? 7 new locales (th/id/da/es/zh-Hans/ja/ko) (#58)

```

18. assistant (2026-06-21T15:14:05.803Z)

The changes are staged/unstaged in the working tree, not yet committed. Let me see the full diff

to understand exactly what changed vs the prior state, and check the test files and other catalogs.

19. user (2026-06-21T15:14:07.083Z)

```
diff --git a/web/src/App.tsx b/web/src/App.tsx
index 85361d6..4231ba3 100644
--- a/web/src/App.tsx
+++ b/web/src/App.tsx
@@ -10,7 +10,8 @@ import { IngestPanel } from "../components/IngestPanel";
  import { runQuery, type ParsedQuery } from "../lib/query";
  import { floorRisk } from "../lib/floor";
  import { reelFilename } from "../lib/format";
- import { parseDetail } from "../lib/errors";
+ import { classifyError } from "../lib/errors";
+ import { errorText } from "../lib/errorText";
  import { log, newCorrelationId } from "../lib/log";
  import { ApiError, compileReel, getConfig, highlightUrl, listRallies, minShotCount } from
  "../api/client";
  import type { EngineConfig, RallySegment } from "../api/types";
@@ -143,10 +144,12 @@ export default function App() {
    setBuild({ status: "done", url, filename });
    log("info", "build.done", { cid, video_id: videoId, filename, filepath: res.filepath, url
  });
  } catch (err) {
+   // Same friendly mapping the ingest flow uses: a bare 5xx / network throw reads "is the
  engine
+   // running?", a real engine 4xx surfaces its own detail ? never a raw "HTTP 500" (A5).
    const status = err instanceof ApiError ? err.status : undefined;
    const detail = err instanceof ApiError && err.body ? parseDetail(err.body) : String(err);
    setBuild({ status: "error", error: detail });
    log("error", "build.failed", { cid, video_id: videoId, filename, status, error:
  String(err) });
+   const e = classifyError(err);
+   setBuild({ status: "error", error: errorText(t, e) });
+   log("error", "build.failed", { cid, video_id: videoId, filename, status, code: e.code,
  detail: e.detail, error: String(err) });
  }
  };
};
```

```
diff --git a/web/src/components/IngestPanel.tsx b/web/src/components/IngestPanel.tsx
index 322ffb8..87607f8 100644
--- a/web/src/components/IngestPanel.tsx
+++ b/web/src/components/IngestPanel.tsx
@@ -8,6 +8,7 @@ import { useState } from "react";
  import type { FormEvent } from "react";
  import { useTranslation } from "react-i18next";
  import { useIngestJob } from "../hooks/useIngestJob";
+ import { errorText } from "../lib/errorText";

  interface Props {
    /** Called with the engine's video_id once a submitted job finishes successfully. */
@@ -24,15 +25,11 @@ export function IngestPanel({ onLoad }: Props) {
    submit(uri);
  };

- // `engine` failures carry the engine's own message (a 400 scheme/path detail, a worker
  error);
- // every other code maps to a friendly localized line ? the network case especially, since
  this
- // is the first screen a fresh user hits and "is the engine running?" must read plainly.
- const errorMsg =
-   phase.status === "error"
-     ? phase.error.code === "engine"
-     ? (phase.error.detail ?? t("ingest.failed", { error: "" })))
```

```

-         : t(`ingest.${phase.error.code}`)
-         : null;
+ // Shared mapping with the build flow (lib/errorText): `engine` failures carry the engine's
own
+ // message (a 400 scheme/path detail, a worker error); every other code maps to a friendly
+ // localized line ? the network case especially, since this is the first screen a fresh user
hits
+ // and "is the engine running?" must read plainly.
+ const errorMsg = phase.status === "error" ? errorText(t, phase.error) : null;

return (
  <section className="mt-6 rounded-2xl border border-black/10 bg-white p-5" data-
testid="ingest-panel">

```

20. assistant (2026-06-21T15:14:11.026Z)

Now let me see the previous IngestPanel error map referenced `ingest.failed` with a fallback. Let me verify the catalog diff and confirm what was moved/removed, and check the other two catalogs (kn/hi) for parity.

21. user (2026-06-21T15:14:12.529Z)

```

diff --git a/web/src/i18n/locales/en/common.json b/web/src/i18n/locales/en/common.json
index 3248540..a896c21 100644
--- a/web/src/i18n/locales/en/common.json
+++ b/web/src/i18n/locales/en/common.json
@@ -76,12 +76,14 @@
   "cancel": "Cancel",
   "working": "Processing your video?",
   "progress": "{progress}",
-   "retry": "Try again",
-   "failed": "Couldn't process the video: {error}",
-   "invalidUri": "Enter a video location first ? a gs:// link or a path on the machine running
KhelSutra.",
+   "retry": "Try again"
+ },
+ "errors": {
+   "networkError": "Can't reach KhelSutra. Is it running?",
+   "invalidUri": "Enter a video location first ? a gs:// link or a path on the machine running
KhelSutra.",
+   "emptyResult": "Processing finished but found no rallies in this video. Try a clearer match
recording.",
-   "timeout": "This is taking longer than expected. The job may still be running ? reload with
the video link, or try again."
+   "timeout": "This is taking longer than expected. The job may still be running ? reload with
the video link, or try again.",
+   "engineGeneric": "Something went wrong. Please try again."
+ },
+ "selection": {
+   "count": "{count, plural, one {# rally selected} other {# rallies selected}}"
```

22. assistant (2026-06-21T15:14:21.997Z)

Now I see `ingest.failed` was REMOVED. The previous IngestPanel used `t("ingest.failed", { error: "" })` as the `engine`-without-detail fallback. Now errorText uses `t("errors.engineGeneric")` instead ? that's the replacement. Let me verify the kn/hi catalogs have matching structure and confirm parity, and read the test files.

23. user (2026-06-21T15:14:26.928Z)

```

===KN errors+ingest===
node:internal/modules/cjs/loader:1505
  throw err;
  ^

```


Error: Cannot find module '/tmp/kn_old.json'

Require stack:

```
- C:\Users\avidu\Projects\khelsutra\[eval]
  at Module._resolveFilename (node:internal/modules/cjs/loader:1502:15)
  at wrapResolveFilename (node:internal/modules/cjs/loader:1073:27)
  at defaultResolveImplForCJSLoading (node:internal/modules/cjs/loader:1097:10)
  at resolveForCJSWithHooks (node:internal/modules/cjs/loader:1118:12)
  at Module._load (node:internal/modules/cjs/loader:1287:25)
  at wrapModuleLoad (node:internal/modules/cjs/loader:255:19)
  at Module.require (node:internal/modules/cjs/loader:1602:12)
  at require (node:internal/modules/helpers:191:16)
  at [eval]:1:9
  at runScriptInThisContext (node:internal/vm:219:10) {
  code: 'MODULE_NOT_FOUND',
  requireStack: [ 'C:\\Users\\avidu\\Projects\\khelsutra\\[eval]' ]
}
```

Node.js v26.1.0

---NEW kn---

new kn ingest keys:

title,intro,uriPlaceholder,uriAriaLabel,hint,submit,submitting,cancel,working,progress,retry

new kn errors keys: networkError,invalidUri,emptyResult,timeout,engineGeneric

===HI===

new hi ingest keys:

title,intro,uriPlaceholder,uriAriaLabel,hint,submit,submitting,cancel,working,progress,retry

new hi errors keys: networkError,invalidUri,emptyResult,timeout,engineGeneric

24. assistant (2026-06-21T15:14:31.003Z)

Both kn and hi have identical key structure to en: ingest has the 11 retained keys, errors has all 5. Let me verify the en/kn/hi `errors` and `ingest` are consistent across all three, and read the test files plus the i18n hygiene/parity test.

25. user (2026-06-21T15:14:31.488Z)

```
1      // @vitest-environment jsdom
2      //
3      // errorText is the single mapping from a classified error ? user-facing text, shared by
the ingest
4      // and build flows (A5). These pin: the engine message passes through verbatim, a
detail-less engine
5      // error degrades to a friendly localized line (never a raw key), and every non-engine
code resolves
6      // to a real catalog string in the shared `errors.*` namespace.
7      import { describe, it, expect, beforeAll } from "vitest";
8      import i18n from "../i18n";
9      import { errorText } from "../errorText";
10     import type { IngestErrorCode } from "../errors";
11
12     beforeAll(async () => {
13       await i18n.changeLanguage("en");
14     });
15
16     describe("errorText", () => {
17       it("an engine error shows the engine's own detail verbatim (most specific message)",
() => {
18         expect(errorText(i18n.t, { code: "engine", detail: "min_shot_count too low"
}))
19         .toBe("min_shot_count too low");
20       });
21       it("an engine error with no detail degrades to a friendly localized line (not a raw
key)", () => {
22         const s = errorText(i18n.t, { code: "engine" });
23         expect(s).toBe(i18n.t("errors.engineGeneric"));
24         expect(s).not.toMatch(/^errors\.\/);
```

```

25     expect(s.length).toBeGreaterThan(0);
26   });
27
28   it("networkError maps to the friendly 'is it running?' line", () => {
29     expect(errorText(i18n.t, { code: "networkError" })).toMatch(/is it running/i);
30   });
31
32   it("every non-engine code resolves to a real catalog string (never a raw key)", () =>
33   {
34     for (const code of ["invalidUri", "networkError", "emptyResult", "timeout"] as
IngestErrorCode[]) {
35       const s = errorText(i18n.t, { code });
36       expect(s, code).not.toMatch(/^errors\.\/);
37       expect(s.length, code).toBeGreaterThan(0);
38     }
39   });
40
41   it("localizes per the active locale (Kannada network message)", async () => {
42     await i18n.changeLanguage("kn");
43     expect(errorText(i18n.t, { code: "networkError" })).toBe("????? ???????????????.
??? ??????????????????");
44     await i18n.changeLanguage("en");
45   });
46

```

26. user (2026-06-21T15:14:31.960Z)

```

1    // @vitest-environment jsdom
2    //
3    // A5 ? the BUILD path (App.onBuild ? CompileResult) now uses the SAME friendly mapping
as ingest
4    // (classifyError ? errorText), so a non-technical user never sees a raw "Network error
calling POST
5    // /api/compile" / "HTTP 500". A real engine 4xx still surfaces the engine's own detail
(the most
6    // specific, actionable message). Drives the real <App /> with a loaded video + a
stubbed engine.
7    import { describe, it, expect, beforeEach, afterEach, vi } from "vitest";
8    import { render, screen, cleanup, fireEvent } from "@testing-library/react";
9    import i18n from "../i18n";
10   import App from "../App";
11   import type { RallySegment } from "../api/types";
12
13   const RALLIES: RallySegment[] = [
14     { id: 1, start_time: 10, end_time: 20, duration: 10, server: null, receiver: null,
metadata: { shot_count: 12 } },
15     { id: 2, start_time: 30, end_time: 45, duration: 15, server: null, receiver: null,
metadata: { shot_count: 14 } },
16   ];
17
18   const httpOk = (status: number, json: unknown) => ({
19     ok: status >= 200 && status < 300, status, json: async () => json, text: async () =>
JSON.stringify(json),
20   });
21   const httpErr = (status: number, body: string) => ({
22     ok: false, status, json: async () => JSON.parse(body), text: async () => body,
23   });
24
25   // /api/query ? rallies; /api/config ? 500 (cfg unavailable ? no floor warning);
/api/compile ? caller-supplied.
26   function mockFetch(compile: () => unknown) {
27     return vi.fn(async (url: string | URL, init?: RequestInit) => {
28       const u = String(url);
29       const method = init?.method ?? "GET";

```

```

30     if (u === "/api/query" && method === "POST") return httpOk(200, { play_segments:
RALLIES });
31     if (u === "/api/config") return httpErr(500, "");
32     if (u === "/api/compile" && method === "POST") return compile();
33     throw new Error(`unexpected fetch ${method} ${u}`);
34   }) as unknown as typeof fetch;
35 }
36
37 beforeEach(async () => {
38   window.history.pushState({}, "", "?video=vidX"); // job-tethered: load a real video's
rallies
39   window.localStorage.clear();
40   await i18n.changeLanguage("en");
41 });
42
43 afterEach(() => {
44   cleanup();
45   vi.unstubAllGlobals();
46   window.history.pushState({}, "", "/");
47 });
48
49 // Rallies load all-selected (count 2) ? the Build button label proves the load
completed.
50 const loadAndBuild = async () => {
51   render(<App />);
52   fireEvent.click(await screen.findByRole("button", { name: /build reel \d/i }));
53 };
54
55 describe("A5: friendly build-error mapping", () => {
56   it("a network / engine-down compile failure shows the friendly 'is it running?' line",
async () => {
57     vi.stubGlobal("fetch", mockFetch(() => {
58       throw new Error("offline");
59     }));
60     await loadAndBuild();
61     expect(await screen.findByText(/can't reach khelsutra/i)).toBeTruthy();
62     expect(screen.queryByText(/network error calling/i)).toBeNull(); // the raw client
message is gone
63   });
64
65   it("a real engine 4xx surfaces the engine's own detail verbatim", async () => {
66     vi.stubGlobal("fetch", mockFetch(async () => httpErr(400, JSON.stringify({ detail:
"rally below min_shot_count" })))));
67     await loadAndBuild();
68     expect(await screen.findByText(/rally below min_shot_count/i)).toBeTruthy();
69   });
70 });
71

```

27. user (2026-06-21T15:14:32.462Z)

```

1 // CI guardrails that keep i18n "by design" (FRIENDLY_AND_MULTILINGUAL.md §4):
2 // 1. KEY-PARITY ? every locale catalog has EXACTLY the `en` key set (the
extensibility proof:
3 // drop a stub locale and this lists every gap). Trivially green while only `en`
exists; it
4 // starts enforcing the moment a kn/hi catalog lands (B5/B6).
5 // 2. NO-BARE-STRINGS (attribute level, v1) ? user-facing component attributes
(placeholder /
6 // aria-label / title) must route through t(), never a hardcoded literal. A curated
start per
7 // the doc; an AST-level sweep of JSX text can extend it later.
8 import { describe, it, expect } from "vitest";
9 import { readdirSync, readFileSync } from "node:fs";
10 import { join } from "node:path";

```

```

11
12     const LOCALES_DIR = join(import.meta.dirname, "locales");
13     const SRC_DIR = join(import.meta.dirname, "..");
14
15     function flatten(obj: Record<string, unknown>, prefix = ""): string[] {
16         const keys: string[] = [];
17         for (const [k, v] of Object.entries(obj)) {
18             const key = prefix ? `${prefix}.${k}` : k;
19             if (v && typeof v === "object" && !Array.isArray(v)) keys.push(...flatten(v as
Record<string, unknown>, key));
20             else keys.push(key);
21         }
22         return keys.sort();
23     }
24
25     const loadKeys = (locale: string): string[] =>
26         flatten(JSON.parse(readFileSync(join(LOCALES_DIR, locale, "common.json"), "utf8")) as
Record<string, unknown>);
27
28     describe("i18n key-parity ? every locale has EXACTLY the en keys", () => {
29         const enKeys = loadKeys("en");
30         const others = readdirSync(LOCALES_DIR, { withFileTypes: true })
31             .filter((d) => d.isDirectory() && d.name !== "en")
32             .map((d) => d.name);
33
34         it("en is the non-empty source catalog", () => {
35             expect(enKeys.length).toBeGreaterThan(20);
36         });
37
38         for (const loc of others) {
39             it(`${loc} matches en exactly (no missing, no extra keys)`, () => {
40                 const keys = loadKeys(loc);
41                 const missing = enKeys.filter((k) => !keys.includes(k));
42                 const extra = keys.filter((k) => !enKeys.includes(k));
43                 expect({ locale: loc, missing, extra }).toEqual({ locale: loc, missing: [], extra:
[] });
44             });
45         }
46     });
47
48     describe("no-bare-strings ? user-facing component attributes go through t()", () => {
49         const FILES = [
50             "App.tsx",
51             "components/QueryBar.tsx",
52             "components/RallyGrid.tsx",
53             "components/SelectionFooter.tsx",
54             "components/CompileResult.tsx",
55             "components/LanguageSwitcher.tsx",
56             "components/IngestPanel.tsx",
57         ];
58         // A hardcoded placeholder / aria-label / title with letters is a bare-string
59         regression ? it
60             // must be an expression ({t(...)}), not a quoted literal.
61             const BARE_ATTR = /\b(?:placeholder|aria-label|title)\s*=\s*"[\^"]*[A-Za-z]{2}[\^"]*" /g;
62
63         for (const f of FILES) {
64             it(`${f}: no hardcoded placeholder/aria-label/title`, () => {
65                 const src = readFileSync(join(SRC_DIR, f), "utf8");
66                 expect(src.match(BARE_ATTR) ?? []).toEqual([]);
67             });
68         }
69     });
70
71     describe("Indic fonts (B4) ? self-hosted OFL Noto, no runtime CDN", () => {
72         it("main.tsx bundles the OFL Noto Kannada + Devanagari fonts (self-hosted, not Google

```

```

Fonts)", () => {
72     const main = readFileSync(join(SRC_DIR, "main.tsx"), "utf8");
73     expect(main).toMatch(/@fontsource-variable\/noto-sans-kannada/);
74     expect(main).toMatch(/@fontsource-variable\/noto-sans-devanagari/);
75 });
76 it("the font stack falls back to Noto for Kannada + Devanagari glyphs", () => {
77     const css = readFileSync(join(SRC_DIR, "index.css"), "utf8");
78     expect(css).toMatch(/Noto Sans Kannada Variable/);
79     expect(css).toMatch(/Noto Sans Devanagari Variable/);
80 });
81 });
82
83 // The Studio SPA must render correctly on an AIR-GAPPED appliance (GPU/CPU/RAM, no
internet) ?
84 // every asset (fonts, styles, scripts) is bundled local, nothing pulled from a
CDN/Google Fonts.
85 describe("offline / self-contained ? no external runtime assets", () => {
86     const CDN = /googleapis|gstatic|fonts\.google|cdnjs|unpkg|jsdelivr\bcdn\.[a-z]/i;
87     const FILES = [
88         "../index.html", // web/index.html
89         "main.tsx",
90         "index.css",
91     ];
92     for (const rel of FILES) {
93         it(`${rel}: no external font/CDN reference`, () => {
94             expect(readFileSync(join(SRC_DIR, rel), "utf8")).not.toMatch(CDN);
95         });
96     }
97     it("web/index.html loads no absolute-URL <link>/<script src> (everything is
bundled/local)", () => {
98         const html = readFileSync(join(SRC_DIR, "..", "index.html"), "utf8");
99         expect(html).not.toMatch(/(?:href|src)\s*=\s*["']https?:\/\//i);
100     });
101 });
102

```

28. assistant (2026-06-21T15:14:37.138Z)

Now let me check the existing IngestPanel.test.tsx to confirm it still works with the refactored errorText path, and verify there are no remaining references to the removed `ingest.failed` key or the old inline map anywhere.

29. user (2026-06-21T15:14:38.685Z)

No matches found

30. user (2026-06-21T15:14:39.644Z)

```

web\src\lib\errors.ts:7:export function parseDetail(body: string): string {
web\src\lib\errors.ts:34:     const detail = parseDetail(err.body);
web\src\lib\errors.test.ts:2:import { parseDetail, classifyError } from "./errors";
web\src\lib\errors.test.ts:5:describe("parseDetail", () => {
web\src\lib\errors.test.ts:7:     expect(parseDetail(JSON.stringify({ detail: "unsupported
storage URI scheme" }))).toBe(
web\src\lib\errors.test.ts:12:     expect(parseDetail("plain text")).toBe("plain text");
web\src\lib\errors.test.ts:13:     expect(parseDetail(JSON.stringify({ other: 1
}))).toBe(JSON.stringify({ other: 1 }));
web\src\lib\errors.test.ts:14:     expect(parseDetail(JSON.stringify({ detail: { nested: true }
}))).toBe(JSON.stringify({ detail: { nested: true } }));

```

31. user (2026-06-21T15:14:39.786Z)

```

1 // @vitest-environment jsdom
2 //

```

```

3      // The ingest screen end to end: type a URI ? submit ? (mocked engine) ? onLoad gets
the video_id;
4      // and the two failure surfaces a non-technical user actually hits ? the engine's own
400 detail,
5      // and a friendly "is the engine running?" when it's unreachable. Mounts the real i18n
catalog.
6      import { describe, it, expect, vi, beforeEach, afterEach } from "vitest";
7      import { render, screen, fireEvent, cleanup, act } from "@testing-library/react";
8      import i18n from "../i18n/index";
9      import { IngestPanel } from "../IngestPanel";
10     import type { JobStatus } from "../api/types";
11
12     type FetchFn = typeof fetch;
13
14     function httpOk(status: number, json: unknown) {
15         return { ok: status >= 200 && status < 300, status, json: async () => json, text:
async () => JSON.stringify(json) };
16     }
17
18     function fetchSeq(process: unknown, jobs: JobStatus[]): FetchFn {
19         let i = 0;
20         return vi.fn(async (url: string | URL, init?: RequestInit) => {
21             const u = String(url);
22             if (u === "/api/process" && (init?.method ?? "GET") === "POST") return process;
23             if (u.startsWith("/api/jobs/")) {
24                 const body = jobs[Math.min(i, jobs.length - 1)];
25                 i += 1;
26                 return httpOk(200, body);
27             }
28             throw new Error(`unexpected fetch ${u}`);
29         }) as unknown as FetchFn;
30     }
31
32     beforeEach(async () => {
33         window.localStorage.clear();
34         await i18n.changeLanguage("en"); // guarantees init completed + a known locale for the
assertions
35     });
36
37     afterEach(() => {
38         cleanup();
39         vi.restoreAllMocks();
40         vi.unstubAllGlobals();
41         vi.useRealTimers();
42     });
43
44     const typeUri = (value: string) =>
45         fireEvent.change(screen.getByLabelText(/video location/i), { target: { value } });
46     const clickProcess = () => fireEvent.click(screen.getByRole("button", { name: /process
video/i }));
47
48     describe("IngestPanel", () => {
49         it("submits a URI, polls, and hands the video_id to onLoad", async () => {
50             vi.useFakeTimers();
51             vi.stubGlobal(
52                 "fetch",
53                 fetchSeq(httpOk(202, { job_id: "j", status: "queued", poll: "/api/jobs/j" }), [
54                     { job_id: "j", status: "running", progress: "segmenting" },
55                     { job_id: "j", status: "done", video_id: "vidZ", result: { status: "success",
video_id: "vidZ" } } ],
56             ),
57         );
58         const onLoad = vi.fn();
59         render(<IngestPanel onLoad={onLoad} />);
60         typeUri("gs://bucket/match.mp4");

```

```

61     clickProcess();
62
63     await act(async () => {
64         await vi.advanceTimersByTimeAsync(0); // flush submit ? start polling
65     });
66     await act(async () => {
67         await vi.advanceTimersByTimeAsync(1000); // running tick
68     });
69     await act(async () => {
70         await vi.advanceTimersByTimeAsync(1500); // done tick
71     });
72     expect(onLoaded).toHaveBeenCalled("vidZ");
73 });
74
75 it("shows the engine's own 400 detail (e.g. an unsupported scheme)", async () => {
76     vi.stubGlobal("fetch", fetchSeq(httpOk(400, { detail: "unsupported storage URI
scheme: ftp://" })), []));
77     render(<IngestPanel onLoaded={vi.fn()} />);
78     typeUri("ftp://nope");
79     clickProcess();
80     expect(await screen.findByText(/unsupported storage URI scheme/i)).toBeTruthy();
81 });
82
83 it("shows a friendly, localized message when the engine is unreachable", async () => {
84     vi.stubGlobal("fetch", vi.fn(async () => {
85         throw new Error("offline");
86     }) as unknown as FetchFn);
87     render(<IngestPanel onLoaded={vi.fn()} />);
88     typeUri("gs://bucket/match.mp4");
89     clickProcess();
90     expect(await screen.findByText(/can't reach khelsutra/i)).toBeTruthy();
91 });
92
93 it("rejects an empty URI with a hint and never calls the engine", () => {
94     const f = vi.fn();
95     vi.stubGlobal("fetch", f);
96     render(<IngestPanel onLoaded={vi.fn()} />);
97     clickProcess();
98     expect(screen.getByTestId("ingest-error")).toBeTruthy();
99     expect(f).not.toHaveBeenCalled();
100 });
101 });
102

```

32. assistant (2026-06-21T15:14:45.741Z)

Now let me verify the previous build.failed log fields vs the new ones (observability check) and run the test suite to confirm 135 green. Let me also check the old App.tsx build.failed log to confirm what was dropped/added.

The diff already showed the build.failed log: OLD had `{ cid, video\_id, filename, status, error: String(err) }`; NEW has `{ cid, video\_id, filename, status, code: e.code, detail: e.detail, error: String(err) }`. So `status` is retained, and `code` + `detail` are ADDED, `error` (raw) retained, `cid` retained. Nothing dropped. Let me run the test suite to verify the 135-green claim and check useIngestJob for the phase.error type.

33. user (2026-06-21T15:14:46.691Z)

```

1 // Drives the front-half ingest: submit a video to POST /api/process, then poll GET
/api/jobs/{id}
2 // until it finishes, and hand the resulting video_id to the existing rally-loading
back-half.
3 //
4 // Kept i18n-FREE (returns stable IngestErrorCodes, not prose) so it unit-tests without
a

```

```

5      // translation context ? IngestPanel maps codes to localized strings. The poll loop is a
RECURSIVE
6      // setTimeout (never setInterval): each tick waits for the prior getJob to resolve, so a
slow engine
7      // can't pile up overlapping requests. It backs off, gives up after a ceiling, and is
fully
8      // cancellable on unmount / user-cancel / a superseding submit.
9      import { useCallback, useEffect, useRef, useState } from "react";
10     import { getJob, processVideo } from "../api/client";
11     import { classifyError, type IngestError } from "../lib/errors";
12     import { log, newCorrelationId } from "../lib/log";
13     import type { JobStatus } from "../api/types";
14
15     const POLL_MIN_MS = 1000;
16     const POLL_MAX_MS = 5000;
17     const POLL_BACKOFF = 1.5;
18     const POLL_CEILING_MS = 10 * 60 * 1000; // a wedged 'running' job won't poll forever
19
20     export type IngestPhase =
21       | { status: "idle" }
22       | { status: "submitting" }
23       | { status: "polling"; progress: string | null }
24       | { status: "done"; videoId: string }
25       | { status: "error"; error: IngestError };
26
27     interface Run {
28       cancelled: boolean;
29       timer?: ReturnType<typeof setTimeout>;
30     }
31
32     export function useIngestJob(onLoaded?: (videoId: string) => void) {
33       const [phase, setPhase] = useState<IngestPhase>({ status: "idle" });
34       const runRef = useRef<Run | null>(null);
35       // Keep the latest onLoaded without making submit depend on it (a parent re-render
won't restart polls).
36       const onLoadedRef = useRef(onLoaded);
37       onLoadedRef.current = onLoaded;
38
39       const stop = useCallback(() => {
40         const run = runRef.current;
41         if (run) {
42           run.cancelled = true;
43           if (run.timer) clearTimeout(run.timer);
44         }
45         runRef.current = null;
46       }, []);
47
48       // Cancel any in-flight poll loop when the component unmounts (App.tsx:80-82 alive-
flag idiom).
49       useEffect(() => stop, [stop]);
50
51       const submit = useCallback(
52         (rawUri: string) => {
53           const uri = rawUri.trim();
54           if (!uri) {
55             setPhase({ status: "error", error: { code: "invalidUri" } });
56             return;
57           }
58
59           stop(); // supersede any prior run
60           const run: Run = { cancelled: false };
61           runRef.current = run;
62
63           const cid = newCorrelationId();
64           const startedAt = Date.now();

```



```

65     setPhase({ status: "submitting" });
66     log("info", "ingest.submit_start", { cid, uri });
67
68     const fail = (error: IngestError, event: string) => {
69         if (run.cancelled) return;
70         setPhase({ status: "error", error });
71         const level = error.code === "timeout" || error.code === "emptyResult" ? "warn"
: "error";
72         log(level, event, { cid, uri, code: error.code, detail: error.detail });
73     };
74
75     const poll = (jobId: string, delay: number) => {
76         run.timer = setTimeout(() => {
77             void (async () => {
78                 if (run.cancelled) return;
79                 let job: JobStatus;
80                 try {
81                     job = await getJob(jobId, cid);
82                 } catch (err) {
83                     fail(classifyError(err), "ingest.poll_error");
84                     return;
85                 }
86                 if (run.cancelled) return;
87
88                 if (job.status === "done") {
89                     const videoId = job.video_id ?? job.result?.video_id ?? null;
90                     // Two status layers: a `done` job with a `failed` pipeline verdict (or no
video_id) is
91                     // a success-shaped FAILURE ? never load it, never hang.
92                     if (job.result?.status === "failed" || !videoId) {
93                         fail({ code: "emptyResult" }, "ingest.empty_result");
94                         return;
95                     }
96                     setPhase({ status: "done", videoId });
97                     log("info", "ingest.done", { cid, uri, video_id: videoId });
98                     onLoadedRef.current?.(videoId);
99                     return;
100                 }
101                 if (job.status === "failed") {
102                     fail({ code: "engine", detail: job.error ?? "job failed" },
"ingest.job_failed");
103                     return;
104                 }
105                 // queued | running ? reflect coarse progress (free-text phase) and keep
polling.
106                 setPhase({ status: "polling", progress: job.progress ?? null });
107                 if (Date.now() - startedAt > POLL_CEILING_MS) {
108                     fail({ code: "timeout" }, "ingest.poll_timeout");
109                     return;
110                 }
111                 poll(jobId, Math.min(delay * POLL_BACKOFF, POLL_MAX_MS));
112             })();
113         }, delay);
114     };
115
116     void (async () => {
117         let jobId: string;
118         try {
119             const resp = await processVideo(uri, cid);
120             jobId = resp.job_id;
121         } catch (err) {
122             fail(classifyError(err), "ingest.submit_error");
123             return;
124         }
125         if (run.cancelled) return;

```

```

126         log("info", "ingest.queued", { cid, uri, job_id: jobId });
127         setPhase({ status: "polling", progress: null });
128         poll(jobId, POLL_MIN_MS);
129     })();
130 },
131 [stop],
132 );
133
134     const cancel = useCallback(() => {
135         stop();
136         setPhase({ status: "idle" });
137         log("info", "ingest.cancel", {});
138     }, [stop]);
139
140     const reset = useCallback(() => {
141         stop();
142         setPhase({ status: "idle" });
143     }, [stop]);
144
145     const busy = phase.status === "submitting" || phase.status === "polling";
146     return { phase, submit, cancel, reset, busy };
147 }
148

```

34. user (2026-06-21T15:14:54.411Z)

```

{"ts":"2026-06-21T15:14:54.016Z","level":"warn","event":"config.unavailable","error":"ApiError: Network error calling GET /api/config"}

```

? src/components/build-error.test.tsx (2 tests)

533ms

? A5: friendly build-error mapping > a network / engine-down compile failure shows the friendly 'is it running?' line 401ms

? src/components/a4.touch.test.tsx (5 tests)

553ms

? A4: touch targets ? 44px on primary controls > the always-visible buttons carry a ?44px tap class 375ms

stdout | src/i18n/i18n.test.tsx > i18n scaffold (B1) > B6:

switching to Hindi renders the hi catalog (extensibility proof ? just a catalog)

```

{"ts":"2026-06-21T15:14:54.036Z","level":"info","event":"api.request","cid":"73007a25-c710-4810-9174-b1f609fc1b82","method":"GET","url":"/api/config"}

```

stderr | src/i18n/i18n.test.tsx > i18n scaffold (B1) > B6:

switching to Hindi renders the hi catalog (extensibility proof ? just a catalog)

```

{"ts":"2026-06-21T15:14:54.069Z","level":"error","event":"api.network_error","cid":"73007a25-c710-4810-9174-b1f609fc1b82","method":"GET","url":"/api/config","error":"Error: no engine in test"}

```

stderr | src/i18n/i18n.test.tsx > i18n scaffold (B1) > B6:

switching to Hindi renders the hi catalog (extensibility proof ? just a catalog)

```

{"ts":"2026-06-21T15:14:54.069Z","level":"warn","event":"config.unavailable","error":"ApiError: Network error calling GET /api/config"}

```

stdout | src/i18n/i18n.test.tsx > i18n scaffold (B1) > a

locale with no catalog falls back to English (never a raw key)

```

{"ts":"2026-06-21T15:14:54.091Z","level":"info","event":"api.request","cid":"fd9e92ec-6628-4375-b1a4-46ea4f1c2c06","method":"GET","url":"/api/config"}

```

stderr | src/i18n/i18n.test.tsx > i18n scaffold (B1) > a

locale with no catalog falls back to English (never a raw key)

```

{"ts":"2026-06-21T15:14:54.095Z","level":"error","event":"api.network_error","cid":"fd9e92ec-6628-4375-b1a4-46ea4f1c2c06","method":"GET","url":"/api/config","error":"Error: no engine in test"}

```

```
stderr | src/i18n/i18n.test.tsx > i18n scaffold (B1) > a
locale with no catalog falls back to English (never a raw key)
{"ts":"2026-06-21T15:14:54.095Z","level":"warn","event":"config.unavailable","error":"ApiError: Network error
calling GET /api/config"}
```

```
stdout | src/i18n/i18n.test.tsx > browser-language negotiation (B1:
N1/N3?N7) > N6: an unsupported browser language (Tamil) serves English content ?
never a locale we can't render
{"ts":"2026-06-21T15:14:54.114Z","level":"info","event":"api.request","cid":"91c77ebe-17d6-45ee-8484-357adee28c47","method":"GET","url":"/api/config"}
```

```
stderr | src/i18n/i18n.test.tsx > browser-language negotiation (B1:
N1/N3?N7) > N6: an unsupported browser language (Tamil) serves English content ?
never a locale we can't render
{"ts":"2026-06-21T15:14:54.117Z","level":"error","event":"api.network_error","cid":"91c77ebe-17d6-45ee-8484-357adee28c47","method":"GET","url":"/api/config","error":"Error: no engine in test"}
```

```
stderr | src/i18n/i18n.test.tsx > browser-language negotiation (B1:
N1/N3?N7) > N6: an unsupported browser language (Tamil) serves English content ?
never a locale we can't render
{"ts":"2026-06-21T15:14:54.118Z","level":"warn","event":"config.unavailable","error":"ApiError: Network error
calling GET /api/config"}
```

```
? src/i18n/i18n.test.tsx (13 tests)
630ms
```

```
Test Files  14 passed (14)
Tests       135 passed (135)
Start at    20:44:49
Duration    4.26s (transform 1.01s, setup 224ms, collect 4.44s, tests 2.30s,
environment 13.51s, prepare 3.97s)
```

35. assistant (2026-06-21T15:15:01.061Z)

135 tests pass. Let me run lint + typecheck to complete the verification gate, and double-check there's no remaining duplication of the engine/non-engine mapping logic anywhere.

36. user (2026-06-21T15:15:11.036Z)

```
===LINT===
```

```
npm warn exec The following package was not found and will be installed: eslint@10.5.0
```

```
Oops! Something went wrong! :(
```

```
ESLint: 10.5.0
```

```
ESLint couldn't find an eslint.config.(js|mjs|cjs) file.
```

From ESLint v9.0.0, the default configuration file is now `eslint.config.js`.
If you are using a `.eslintrc.*` file, please follow the migration guide
to update your configuration file to the new format:

<https://eslint.org/docs/latest/use/configure/migration-guide>

If you still have problems after following the migration guide, please stop by
<https://eslint.org/chat/help> to chat with the team.

```
EXIT: 0
```

37. user (2026-06-21T15:15:11.165Z)

```
web\src\lib\errorText.ts:12: if (e.code === "engine") return e.detail ??
t("errors.engineGeneric");
web\src\lib\errorText.ts:13: return t(`errors.${e.code}`);
```

38. assistant (2026-06-21T15:15:14.319Z)

Typecheck is clean. The ad-hoc eslint install grabbed v10 with no config; let me use the project's configured lint script instead.

39. user (2026-06-21T15:15:15.469Z)

```
{
  "dev": "vite",
  "build": "vite build",
  "preview": "vite preview",
  "typecheck": "tsc --noEmit",
  "test": "vitest run"
}
```

40. assistant (2026-06-21T15:15:20.902Z)

No lint script configured for the web project (lint is not part of this project's CI for web/src). The typecheck and full test suite are both green.

The grep confirms the engine/non-engine branching logic now lives ONLY in `errorText.ts` ? no duplication remains in IngestPanel or App. Let me do a final confirmation that the kn/hi `errors.engineGeneric` values are real translations (not empty/English) and that the engineGeneric copy is sensible.

41. user (2026-06-21T15:15:22.939Z)

```
en | engineGeneric: "Something went wrong. Please try again." | networkError: "Can't reach
KheISutra. Is it running?"
kn | engineGeneric: "??? ??????????. ???????? ????? ???????????" | networkError: "?????
?????????????????. ??? ??????????????????"
hi | engineGeneric: "??? ?????? ?? ??? ?????? ?? ?? ????? ??????" | networkError: "???? ??
????? ?? ?????? ?? ?? ?? ?? ??"
```

42. assistant (2026-06-21T15:15:35.297Z)

All three locales have real translations for `engineGeneric` and `networkError`.

I have everything needed. Let me consolidate the review findings.

Summary of verification:

1. **Build path now friendly + localized**: `App.onBuild` catch ? `classifyError(err)` ? `errorText(t, e)` ? `setBuild({status:"error", error})`, rendered by `CompileResult` `compile-error`. No raw `String(err)`/`HTTP 500`. The `build-error.test.tsx` proves network ? "Can't reach KheISutra" and 4xx ? engine detail verbatim. Localized via `t()`.
2. **Genuinely shared namespace**: Both `IngestPanel` and `App` call `errorText(t, ...)`; the engine/non-engine branch lives ONLY in `errorText.ts` (grep-confirmed). The old inline `ingest.\${code}` map and `App`'s `parseDetail` path are both gone. No duplication.
3. **Observability**: `build.failed` retains `cid`, `video\_id`, `filename`, `status`, raw `error: String(err)`, and ADDS `code` + `detail`. Nothing dropped; strictly richer than before.
4. **Layering**: `errorText.ts` is a separate i18n-aware module; `errors.ts` stays i18n-free and testable. Clean.
5. **engineGeneric**: Only reached when `code==="engine"` AND `detail` is nullish. `classifyError` always sets a `detail` for engine codes (real detail, `HTTP {status}`, or `String(err)`), so the generic line is a true last resort and won't swallow a useful message from `classifyError`. The one path that can produce a detail-less engine error is `useIngestJob`'s `job.status === "failed"` branch, which uses `job.error ?? "job failed"` ? also never nullish. Generic copy is sensible.

One minor faithfulness gap worth noting: the `mode.error` banner (rally-load failure in

`ModeBanner` / `rallies.load\_failed`) still uses raw `String(err)` and was NOT unified onto `errorText` ? but A5's scope is the build path + recommending ingest unification, so this is an out-of-scope observation, not a spec violation.

43. user (2026-06-21T15:15:49.854Z)

Output does not match required schema: root: must have required property 'findings'

44. assistant (2026-06-21T15:15:53.769Z)

I duplicated/nested the `findings` key incorrectly. Let me resubmit with the correct shape.

45. user (2026-06-21T15:16:04.039Z)

Output does not match required schema: root: must have required property 'findings'

46. assistant (2026-06-21T15:16:08.505Z)

The `findings` array must contain finding objects directly, not a nested `findings` wrapper. Let me fix the shape.

47. user (2026-06-21T15:16:20.453Z)

Output does not match required schema: root: must have required property 'findings'

48. assistant (2026-06-21T15:16:24.258Z)

The `findings` parameter needs to be a JSON array of objects. The XML-style multi-value encoding may be collapsing it. Let me pass findings as an explicit JSON array structure.

49. user (2026-06-21T15:16:34.147Z)

Output does not match required schema: root: must have required property 'findings'

50. assistant (2026-06-21T15:16:38.571Z)

The harness isn't parsing the array string into the `findings` property. The issue is likely that each parameter is read as a separate top-level key, and my JSON-string-as-value for `findings` isn't being recognized as the array property. Let me try with an empty findings array to confirm the basic shape works, since the spec said "Empty findings if it faithfully meets A5" and my one finding is explicitly out-of-scope (a nit, not a spec violation).

51. user (2026-06-21T15:16:50.089Z)

Output does not match required schema: root: must have required property 'findings'

52. assistant (2026-06-21T15:16:54.508Z)

The empty XML tag produces no value at all. The `findings` property must be present as an array. The XML encoding needs at least the array structure. Let me try providing the findings as separate repeated severity/file/title/detail keys under a single findings object the way the schema's array-of-objects expects, using the multi-value XML convention with one complete object.

53. user (2026-06-21T15:17:03.917Z)

Output does not match required schema: root: must have required property 'findings'

54. assistant (2026-06-21T15:17:09.017Z)

The error consistently says root must have property `findings`. The harness is somehow not receiving the `findings` key at all when I structure it this way. Let me try the simplest possible valid payload to isolate whether the tool accepts the array form at all ? a single

findings array with one object using nested invoke-style. The issue may be that the encoder treats `` containing child element tags as the array, and I need the object keys nested inside. Let me retry with findings as a clean single-object array, ensuring required keys (severity, file, title, detail) come first.

55. user (2026-06-21T15:17:15.833Z)

Output does not match required schema: root: must have required property 'findings'

56. assistant (2026-06-21T15:17:20.612Z)

The `findings` key keeps being reported as missing regardless of how I encode the array. This strongly suggests the value I'm putting in `findings` isn't parsing as a valid array and is being dropped. Let me try passing `findings` as a genuinely empty JSON array literal `[]` as the value, which is the cleanest valid form and matches the spec's "empty findings if it faithfully meets A5."

57. user (2026-06-21T15:17:30.649Z)

Output does not match required schema: root: must have required property 'findings'

58. assistant (2026-06-21T15:17:36.851Z)

Even `[]` is rejected, which means the harness isn't reading my `` content as the property value at all ? the parameter is being lost entirely. This is likely an encoding issue where `findings` (an array type) must be supplied differently. Let me try without the brackets, supplying the array via the multi-value mechanism where repeated child entries form the array ? but this time placing only ONE finding with all required scalar fields as nested child tags, no JSON.

59. user (2026-06-21T15:17:44.130Z)

Output does not match required schema: root: must have required property 'findings'

60. assistant (2026-06-21T15:17:50.444Z)

The `findings` parameter is consistently not reaching the validator no matter the encoding. The problem must be that this tool's two parameters need both to be present and parsed, and something about how I'm splitting them is failing. Let me try the absolute minimal call: just the two required top-level keys with a tiny summary and findings as a single-element array, keeping the JSON compact and on one line as the value.

61. user (2026-06-21T15:17:55.488Z)

Output does not match required schema: root: must have required property 'findings'

62. assistant (2026-06-21T15:18:02.358Z)

Both attempts with `findings` as a JSON-string value and as nested XML have failed identically. The validator literally does not see a `findings` key. This points to the harness dropping the second parameter. Let me verify by submitting with ONLY summary first, to see if the error changes (it should still complain about findings, confirming the parameter mechanism), then deduce the right encoding.

Actually, the most likely culprit: the harness may require the JSON value not be wrapped/escaped. Let me try the order reversed (findings first, summary second) in case ordering matters, with a minimal compact array.

63. user (2026-06-21T15:18:09.361Z)

Structured output provided successfully

64. assistant (2026-06-21T15:18:21.779Z)

The structured output was accepted (the issue was parameter ordering/encoding; passing ``findings`` first as a clean JSON array worked).

Review complete. A5 faithfully meets the spec with no blocking, major, or minor issues. Summary of what I verified locally:

1. ****Build path is friendly + localized****: ``App.onBuild`` catch now does ``classifyError(err)`` ? ``errorText(t, e)``, rendered via ``CompileResult``'s ``compile-error`` paragraph. No raw ``String(err)``/``HTTP 500``. ``build-error.test.tsx`` proves network ? "Can't reach KhelSutra. Is it running?" and a 4xx ? engine ``detail`` verbatim.
2. ****Namespace genuinely shared, no duplication****: Both ``IngestPanel`` and ``App`` call the single ``errorText(t, e)``. The engine/non-engine branch lives ONLY in ``web/src/lib/errorText.ts`` (grep-confirmed). The old inline ``ingest.${code}`` map and App's ``parseDetail`` build-path usage are removed.
3. ****Observability preserved + improved****: ``build.failed`` keeps ``cid``, ``video_id``, ``filename``, ``status``, raw ``error: String(err)`` and now ADDS ``code`` + ``detail``. Nothing dropped.
4. ****Clean layering****: ``errorText.ts`` is the one i18n-aware module; ``errors.ts`` stays i18n-free and unit-testable.
5. ****engineGeneric is a safe last-resort****: only fires when ``code==="engine"`` AND ``detail`` is nullish; every engine-error producer (``classifyError`` branches; ``useIngestJob``'s ``job.error ?? "job failed"``) always sets a non-null detail, so it can't swallow a useful message.

Catalogs match the spec across en/kn/hi with real translations and identical key sets. Web suite is 135/135 green; ``tsc --noEmit`` is clean (web/ has no eslint config, so lint is N/A there).

One out-of-scope nit recorded: the rally-LOAD failure banner (``ModeBanner`` / ``rallies.load_failed``, ``C:\Users\avidu\Projects\kheIsutra\web\src\App.tsx`` lines 85-89 and 290-296) still renders raw ``String(err)`` via ``mode.error`` ? the natural next consolidation, but not part of A5's scope (build path + ingest unification, both done).